الجامعة المصرية اليابانية للعلوم والتكنولوجيا

E-JUST

Egypt - Japan University of Science and Technology

エジプト日本科学技術大学

Egypt-Japan University of Science and Technology

Mechatronics and Robotics Engineering

Mobile Robot Project

Submitted by:

	Student Name	Student ID
1.	Shams Eldeen Anter	120220323
2.	Ahmed Abdelraheem	120220122
3.	Shahd Mohamed Ali	120220190
4.	Mahmoud Mohamed Ali	120220302
5.	Reham Ahmed Mohamed	120220326
6.	Anas Osama Ali Dorgham	120210156
7.	Adhm Yasser Ahmed	120220127
8.	Abdelrahman Hossam	120220333
9.	Mahmoud Taha Mahmoud	120220275
10.	Mohammed gamal	120220341

Course: Mobile Robots
Instructor: Dr. Haitham Elhussieny

December 30, 2025

Contents

1 Introduction **3**

2 Kinematic Analysis **3**
2.1 Robot Dimensions and Wheel Configuration 4
2.2 Inverse Kinematics Equations 5

3 Design Choices **5**

4 Mechanical Layout **6**

5 System Architecture **7**
5.1 Interfaces 7

6 Software Implementation **7**
6.1 URDF Modeling and Simulation 7
6.1.1 Holonomic Control Plugin 8
6.1.2 Procedural Wheel Generation 8
6.2 Inverse Kinematics Implementation 9
6.3 Wireless Communication Protocol 10
6.4 Embedded Firmware Implementation 10
6.4.1 ESP32: Wireless Bridge and I2C Master 11
6.5 Visual Synchronization (RViz & Gazebo) 13

7 Conclusion **13**

List of Figures

1 Top view of the mobile robot with dimensions 4
2 Robot Configuration 6
3 High-level architecture. Both RViz2 and Gazebo are driven by the same open-loop integrated pose. 7
4 The URDF model spawned in Gazebo. The rollers are generated procedurally to match the 45° mecanum vectors. 8

1 Introduction

This project focuses on the design, implementation, and control of an omnidirectional mobile robot. The primary objective was to develop a system capable of holonomic motion moving instantaneously in any direction (V_x, V_y) and rotating (ω_z) simultaneously while integrating a robust distributed control architecture to satisfy the requirements of kinematic analysis and real-world control. To achieve this omnidirectional mobility, we selected a Four-Wheel Mecanum Configuration over the alternative Swedish 90° setup. The Mecanum wheel features passive rollers mounted at 45° angles, allowing the robot to generate force vectors diagonally for complex maneuvers like lateral strafing. Mechanically, this four-wheel setup offers superior static stability and supports a balanced payload distribution for the onboard electronics.

Unlike standard differential drive systems, our robot utilizes a distributed processing architecture that separates high-level kinematic planning from low-level motor control. Although standard guidelines often suggest a direct USB connection, our implementation enhances this by using a wireless architecture. The control logic resides on a host laptop running ROS 2, where a path-planning algorithm generates a target JSON array containing the required angular velocities. This data is transmitted wirelessly via WiFi to an ESP32 microcontroller, which serves as a communication bridge. The ESP32 parses the data and transmits it via I2C to an Arduino Uno, which acts as a dedicated low-level actuator generating PWM signals for the motors through dual L293 drivers.

To address the challenges of open-loop control inherent in this setup, an MPU6050 IMU (Inertial Measurement Unit) was integrated as an optional sensor. While the current control loop relies on pre-calculated kinematic models, the IMU provides real-time data on the robot's heading and rotational acceleration. This sensor data is utilized to monitor performance and calibrate the system against drift, validating the accuracy of the kinematic algorithms during complex maneuvers.

2 Kinematic Analysis

We used a four-Mecanum wheel configuration where all wheels are fixed. To model the kinematics of the robot, we need to study the constraints on the motion expressed by the wheel type.

Two constraints for each wheel type:

Rolling constraint: The wheel must roll when motion takes place in the appropriate direction.

$$[\sin(\alpha+\beta+\gamma) \quad -\cos(\alpha+\beta+\gamma) \quad -l\cos(\beta+\gamma)] \cdot \dot{\zeta}_R - r\dot{\phi}\cos(\gamma) = 0$$

No-slippage constraint: The wheel cannot slide laterally.

$$[\cos(\alpha+\beta+\gamma) \quad \sin(\alpha+\beta+\gamma) \quad -l\sin(\beta+\gamma)] \cdot R(\theta)\dot{\zeta}_I - r\dot{\phi}\sin\gamma - r_{sw}\dot{\phi}_{sw} = 0$$

Where:

- l, α: Defines the rotation axis position.
- β: The wheel angle (fixed).
- γ: Rollers angle.
- r: Wheel radius ($r = 4$).

2.1 Robot Dimensions and Wheel Configuration

Figure 1 below illustrates the physical dimensions of our robot kit, including the wheel placement relative to the center.

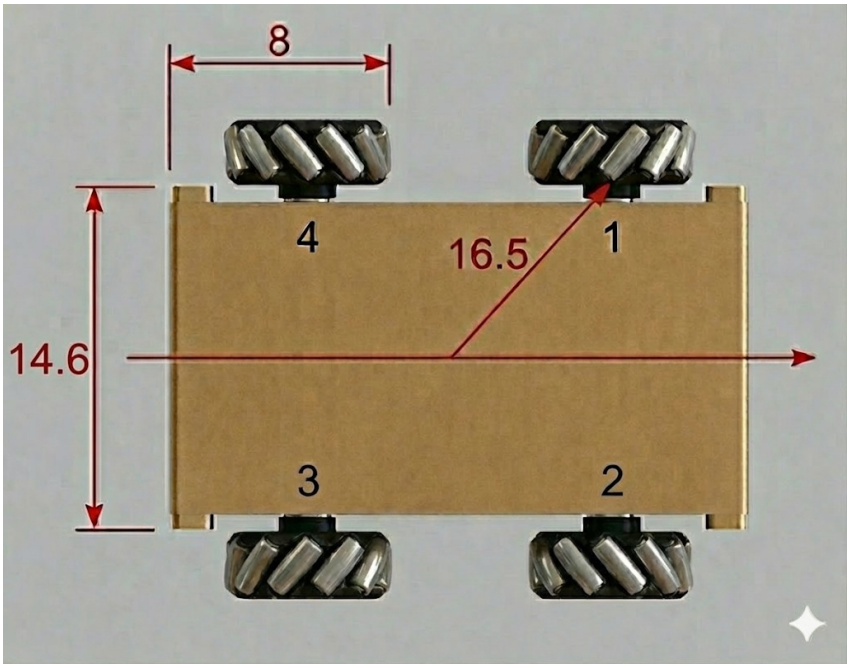

Figure 1: Top view of the mobile robot with dimensions

Based on the dimensions of our kit shown above:

$$x = 8, \quad y = 7.3, \quad \text{Length from the robot center to the wheel (l)} \ = 16.5$$

We analyze the angles α (Position) and β (Roller) for each wheel:

For First Wheel:

$$\alpha_1 = 43.5^\circ, \quad \beta_1 = -45^\circ$$

For Second Wheel:

$$\alpha_2 = 133.5^\circ, \quad \beta_2 = -135^\circ$$

For Third Wheel:

$$\alpha_3 = 223.5^\circ, \quad \beta_3 = -45^\circ$$

For Fourth Wheel:

$$\alpha_4 = 313.5^\circ, \quad \beta_4 = 135^\circ$$

2.2 Inverse Kinematics Equations

Substituting the parameters into the rolling constraint equation, we obtain the relationship between wheel velocities and robot body velocities.
Matrix Form: Using the geometric factor derived from our dimensions (16.5):

$$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \\ \dot{\phi}_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & -16.5 \\ -\frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 16.5 \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 16.5 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & -16.5 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$$

Scalar Equations: Simplifying the matrix yields the following equations:

$$\begin{aligned} \dot{\phi}_1 &= \frac{1}{4}\left(\dot{x} - \dot{y} - 23.33\dot{\theta}\right) \\ \dot{\phi}_2 &= \frac{1}{4}\left(-\dot{x} - \dot{y} + 23.33\dot{\theta}\right) \\ \dot{\phi}_3 &= \frac{1}{4}\left(\dot{x} - \dot{y} + 23.33\dot{\theta}\right) \\ \dot{\phi}_4 &= \frac{1}{4}\left(-\dot{x} - \dot{y} - 23.33\dot{\theta}\right) \end{aligned}$$

3 Design Choices

Designing a mobile robot is a balance between mechanical capability, control complexity, and power management. To meet our goal of a wireless, omnidirectional platform, we selected the following components:

- **Mecanum Wheels (45°):** We specifically chose wheels with rollers mounted at a 45° angle. This unique geometry is the key to our robot's agility. It allows for "holonomic" motion, meaning the robot can strafe sideways or move diagonally without needing to turn its body. This capability simplifies our chassis design significantly, as we do not need complex steering linkages or servos to change direction.

- **4 DC Gear Motors:** To take full advantage of the Mecanum wheels, we require independent control over each wheel. We selected four DC gear motors because they offer the high torque necessary to move the chassis while maintaining the precise speed control required by our kinematic equations. Each motor acts as an independent actuator in our vector calculations.

- **ESP 32 Microcontroller:** For the "brain" of the robot, we moved away from standard wired controllers and chose the ESP 32. We selected this module for its built-in Wi-Fi and Bluetooth capabilities. It serves as a wireless bridge, receiving velocity commands from our ROS 2 system and translating them for the hardware. This eliminates the traditional USB tether, giving the robot true freedom of movement.

- **Motor Shield L293:** Electronics are sensitive, and motors are power-hungry. To bridge this gap, we used the L293 Motor Shield. This component protects the delicate logic circuits of the ESP 32 from the high currents required by the motors. It also allows us to use Pulse Width Modulation (PWM) to finely tune the speed of the robot, ensuring smooth acceleration and accurate maneuvering.

4 Mechanical Layout

When we built the robot, we wanted to keep things organized. As you can see in the photo, we built it like a double-decker bus.

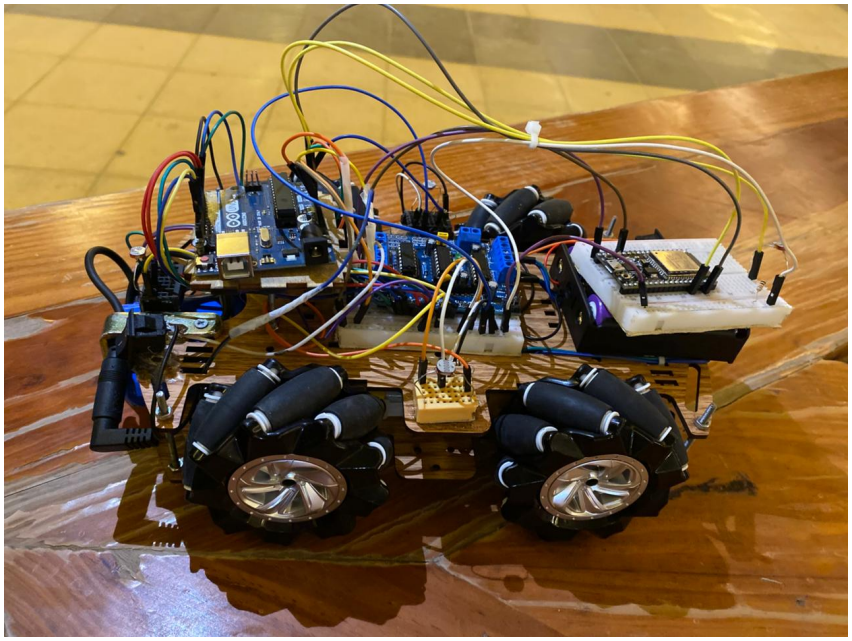

Figure 2: Robot Configuration

- **Two Separate Layers:** We split the robot into two levels. The bottom blue plate is for the moving parts it holds the four motors securely. The top wooden plate is for the "brains" it keeps the wires and chips safe from the spinning motors below.
- **Where We Put the Electronics:** On the top layer, we placed the Arduino and the Motor Shield on the left side. This is right in the middle of the four wheels, so the wires from the motors can reach it easily. On the right side, we put the ESP 32 on a breadboard. We raised it up a little bit to make sure it gets a good Wi-Fi signal.
- **Balancing the Weight:** We put the heavy battery pack at the very back of the top plate. This helps balance the robot so it doesn't tip over when it moves fast.
- **Wiring It Up:** We used jumper wires to connect the ESP 32 (which receives the wireless signals) to the Arduino (Controller). The wires from the motors come up from the bottom layer and plug into the shield on top. This keeps everything neat and easy to fix if something goes wrong.

5 System Architecture

Figure 3 summarizes the runtime data flow. A single ROS 2 node consumes `/cmd_vel` and produces multiple outputs used by RViz2 and Gazebo. The design deliberately avoids having Gazebo compute base motion from physics or base plugins, because the project requires deterministic synchronization with the open-loop integrator.

Architecture diagram placeholder.
`/cmd_vel` → Open-loop Node → (TF, /odom, /joint_states, JSON debug) and Gazebo model follower.

Figure 3: High-level architecture. Both RViz2 and Gazebo are driven by the same open-loop integrated pose.

5.1 Interfaces

The principal interfaces are shown in Table 1. The `/tf` transform `odom` → `base_link` is the primary driver for RViz2 motion. The Gazebo follower uses a Gazebo service to set the model pose to the same (x, y, θ) produced by the integrator.

Name	Type	Role
`/cmd_vel`	`geometry_msgs/Twist`	Holonomic velocity commands
`/tf`	TF	Integrated pose: `odom` → `base_link`
`/odom`	`nav_msgs/Odometry`	Optional open-loop odometry message
`/joint_states`	`sensor_msgs/JointState`	Wheel rotation visualization in RViz2
Gazebo service	`gazebo_msgs`	Set model/entity state to match open-loop pose

Table 1: Main interfaces used in the project.

6 Software Implementation

The control logic is centralized in a custom ROS 2 node named `CmdVelToESP32WithGazeboFollower`. This Python-based node serves as the critical bridge between the high-level path planning (ROS 2) and the low-level hardware actuation (ESP32/Arduino). It performs three parallel tasks: computing inverse kinematics, maintaining an open-loop odometry estimate, and managing the wireless TCP/IP connection.

6.1 URDF Modeling and Simulation

To validate our kinematic logic before hardware deployment, we developed a parametric simulation using the Unified Robot Description Format (URDF) and Xacro (XML Macros). This approach allowed us to define physical properties—such as chassis mass (2.0 kg) and wheel radius (0.04 m)—as variables, ensuring the model matches the physical build.

Figure 4 visualizes the resulting model spawned in the Gazebo environment.

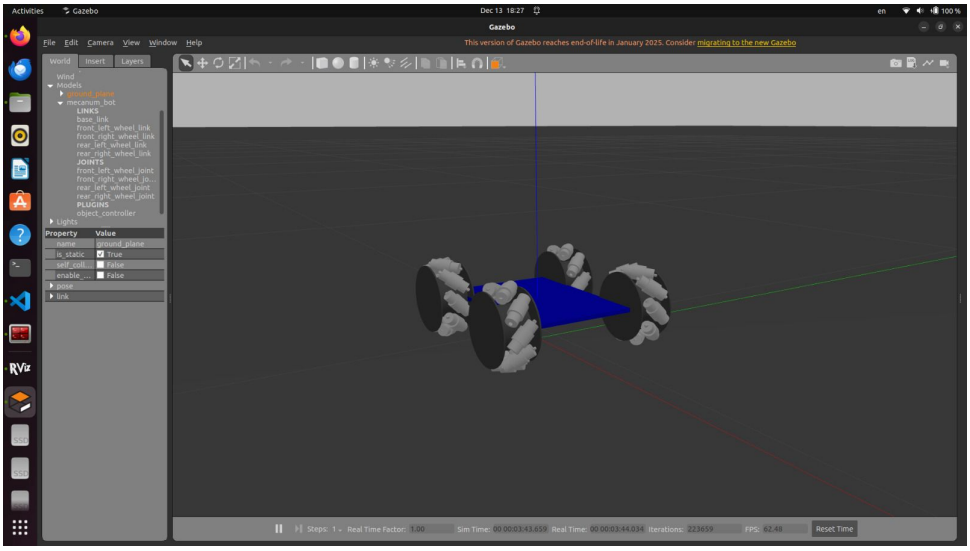

Figure 4: The URDF model spawned in Gazebo. The rollers are generated procedurally to match the 45° mecanum vectors.

6.1.1 Holonomic Control Plugin

In the main robot description file, the critical component is the Gazebo plugin. Standard differential drive plugins cannot handle lateral movement. We utilized the `libgazebo_ros_planar_move.` plugin, which accepts `cmd_vel` and moves the robot holonomically (x, y, θ) without requiring complex friction simulation for the rollers.

Listing 1: Main Robot Xacro: Plugin Configuration.

```
<robot name="mecanum_bot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:property name="chassis_length" value="0.268"/>
  <xacro:property name="chassis_mass" value="2.0"/>

  <gazebo>
    <plugin name="object_controller" filename="libgazebo_ros_planar_move.so
      <ros>
        <remapping>cmd_vel:=cmd_vel</remapping>
        <remapping>odom:=odom</remapping>
      </ros>
      <update_rate>100</update_rate>
      <publish_odom>true</publish_odom>
      <robot_base_frame>base_link</robot_base_frame>
    </plugin>
  </gazebo>
</robot>
```

6.1.2 Procedural Wheel Generation

Modeling a Mecanum wheel manually is inefficient due to the multiple rollers. We implemented a recursive Xacro macro to generate the wheel geometry. As shown in Listing

2, the `create_rollers` macro calls itself until all 9 rollers are placed around the hub, automatically calculating the correct angle for each.

Listing 2: Recursive Macro for Roller Generation.

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:property name="num_rollers" value="9"/>
  <xacro:property name="wheel_radius" value="0.04"/>

  <xacro:macro name="create_rollers" params="name parent_link roller_angle
    <xacro:if value="${count > 0}">
      <xacro:property name="angle" value="${(count-1) * (2*pi/num_rollers)}

      <xacro:roller_visual name="${name}_${count}" parent_link="${parent_li
                           angle="${angle}" roller_angle="${roller_angle}"/

      <xacro:create_rollers name="${name}" parent_link="${parent_link}"
                            roller_angle="${roller_angle}" count="${count-
    </xacro:if>
  </xacro:macro>

</robot>
```

6.2 Inverse Kinematics Implementation

The node subscribes to the `/cmd_vel` topic. Upon receiving a linear velocity vector (V_x, V_y) and an angular velocity (ω_z), it applies the matrix equations derived in Section 2. The implementation ensures that the robot's physical dimensions (l_x, l_y) and wheel radius (r) are parameterized, allowing for easy tuning.

Listing 3 demonstrates the translation of the mathematical model into Python code.

```python
def mecanum_inverse_kinematics(self, vx: float, vy: float, wz: float)
    -> List[float]:
    """
    Returns wheel angular speeds [rad/s] in order [FL, FR, RL, RR].
    Based on the derived Rolling Constraints.
    """
    r = float(self.get_parameter("wheel_radius").value)
    lx = float(self.get_parameter("lx").value)
    ly = float(self.get_parameter("ly").value)

    # Geometric factor (alpha + beta)
    a = lx + ly

    # X-roller configuration calculations
    w_fl = (vx - vy - a * wz) / r
    w_fr = (vx + vy + a * wz) / r
    w_rl = (vx + vy - a * wz) / r
    w_rr = (vx - vy + a * wz) / r

    return [w_fl, w_fr, w_rl, w_rr]
```

Listing 3: Inverse Kinematics Logic from the ROS Node.

6.3 Wireless Communication Protocol

To eliminate the need for a wired USB tether, this node acts as a TCP client. It connects to the ESP32 (configured as a TCP server) on a static IP address, specifically `172.30.101.210` at port `5000`.

The communication loop runs at a fixed frequency (defaults to 30Hz) to ensure smooth motion. At each interval, the calculated wheel velocities are converted into PWM values (0-255). These values are then packaged into a JSON object. We utilized JSON for the transmission protocol because it provides a structured, human-readable format that is easily parsed by the ESP32.

The JSON payload structure is defined as:

$$\{"u" : "\text{pwm}", "w" : [w_{FL}, w_{FR}, w_{RL}, w_{RR}], "t" : \text{timestamp}\}$$

Listing 4 shows the connection handling and data serialization logic.

```python
def on_send_timer(self) -> None:
    # Ensure TCP connection is active
    self._connect_if_needed()

    # Get current command and calculate kinematics
    vx, vy, wz = self._get_cmd()
    physical_wheels = self.mecanum_inverse_kinematics(vx, vy, wz)

    # Map to output order and convert to PWM
    wheels_out = self.apply_output_mapping(physical_wheels)
    pwm_values = self.wheels_to_pwm(wheels_out)

    # Construct the JSON Payload
    now_wall = time.time()
    payload = {
        "u": "pwm",
        "w": pwm_values,
        "t": now_wall
    }

    # Serialize to string and send over socket
    self._send_line(json.dumps(payload))

def _send_line(self, line: str) -> None:
    if self.sock is None:
        return
    try:
        # Send data with newline termination for ESP32 parsing
        self.sock.sendall((line + "\n").encode("utf-8"))
    except Exception as e:
        self.get_logger().warn(f"Send failed: {e}")
        self._close_socket()
```

Listing 4: TCP Connection and JSON Serialization.

6.4 Embedded Firmware Implementation

The low-level control is split between an ESP32 (Communications) and an Arduino Uno (Actuation). This separation ensures that the heavy processing required for WiFi and JSON parsing does not interrupt the precise PWM generation required for the motors.

6.4.1 ESP32: Wireless Bridge and I2C Master

The ESP32 is configured as a TCP server on port 5000. It waits for a connection from the ROS 2 node. Upon receiving a newline-terminated JSON string, it performs two critical tasks:

1. **Deserialization:** It parses the JSON data into a `RobotAngles` structure. This ensures that the four velocity values (T_1 to T_4) are correctly mapped.
2. **Binary I2C Transmission:** Instead of sending raw text to the Arduino, the ESP32 packs the four float values into a binary structure and transmits the entire memory block (16 bytes) via I2C to the slave device (Address 8). This method is significantly faster and more robust than parsing strings on the Arduino.

Listing 5 shows the complete implementation used on the ESP32.

Listing 5: ESP32 Firmware: TCP Server and Binary I2C Transmitter.

```
#include <WiFi.h>
#include <ArduinoJson.h>
#include <Wire.h>

const char* ssid = "YOUR_SSID";        // Configured network credentials
const char* password = "YOUR_PASS";
WiFiServer server(5000);

#define I2C_SLAVE_ADDR 8

// Structure must match Arduino receiver exactly
// Sends 4 floats = 16 bytes of data
struct RobotAngles {
  float t1;
  float t2;
  float t3;
  float t4;
};

RobotAngles angles;

void setup() {
  Serial.begin(115200);
  Wire.begin(); // ESP32 is Master

  // Establish WiFi Connection
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting...");
  }
  Serial.println("Connected! IP: " + WiFi.localIP().toString());
  server.begin();
```

```
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    Serial.println("New Client connected.");

    while (client.connected()) {
      if (client.available()) {
        // Read incoming JSON string (terminated by newline)
        String data = client.readStringUntil('\n');
        data.trim();
        if (data.length() == 0) continue;

        StaticJsonDocument<200> doc;
        DeserializationError error = deserializeJson(doc, data);

        if (!error) {
          // 1. Pack parsed JSON data into Struct
          angles.t1 = doc[0];
          angles.t2 = doc[1];
          angles.t3 = doc[2];
          angles.t4 = doc[3];

          // 2. Debug Logging
          Serial.println("\n--- Sending to Arduino ---");
          Serial.print("T1: "); Serial.println(angles.t1);
          Serial.print("T2: "); Serial.println(angles.t2);
          Serial.print("T3: "); Serial.println(angles.t3);
          Serial.print("T4: "); Serial.println(angles.t4);

          // 3. Send Binary Data over I2C
          Wire.beginTransmission(I2C_SLAVE_ADDR);
          // Cast struct pointer to uint8_t* for raw byte transfer
          Wire.write((uint8_t *)&angles, sizeof(angles));
          byte i2cError = Wire.endTransmission();

          if (i2cError == 0) {
            Serial.println("Result: Success");
          } else {
            Serial.print("Result: FAILED (Error ");
            Serial.print(i2cError);
            Serial.println(")");
          }
        }
      }
    }
```

```
    client.stop();
  }
}
```

6.5 Visual Synchronization (RViz & Gazebo)

In addition to driving the physical robot, the node integrates the velocity commands over time to estimate the robot's pose (x, y, θ) in an open-loop fashion. It broadcasts this pose to the `/tf` topic and publishes `/joint_states`. This allows the operator to visualize the robot's intended path and wheel rotation in RViz2 in real-time, even without external odometry sensors.

7 Conclusion

In this project, we successfully designed, simulated, and implemented a four-wheel omnidirectional mobile robot utilizing Mecanum wheels. The primary objective was to achieve true holonomic motion, allowing the platform to translate instantaneously in any direction while controlling its orientation. Our results demonstrate that the derived kinematic constraints were accurate. The robot is capable of performing complex maneuvers—such as lateral strafing and diagonal translation—verifying the effectiveness of the X-configuration roller layout. A key achievement of this system is the distributed control architecture. By decoupling the high-level path planning (ROS 2 on Host PC) from the low-level motor actuation (Arduino), and bridging them via a wireless TCP connection (ESP32), we eliminated the limitations of a wired tether. The use of binary I2C transmission between the microcontrollers ensured that this wireless freedom did not come at the cost of control latency.